

CSCE 608 DATABASE SYSTEMS · MODULE 10 · LECTURE NOTES

Multi-Version Concurrency Control

Readers that never block, and the garbage they create.

MODULE OUTCOMES

- Explain how MVCC makes readers non-blocking.
- Explain snapshot isolation and what it permits.
- Explain write skew and why snapshot isolation allows it.
- Explain version storage and garbage collection.
- Explain serializable snapshot isolation.

1 The idea

READERS NEVER BLOCK WRITERS; WRITERS NEVER BLOCK READERS

Under two-phase locking, a long-running read holds shared locks for its entire duration, and every writer touching those rows waits. **A single analytical query can stall an entire transactional workload**, which is intolerable for a system that must serve both.

MVCC keeps multiple versions of each row. A writer does not overwrite; it creates a new version. Readers continue to see whichever version was current when they began.

So no reader ever waits for a writer, and no writer ever waits for a reader. Only writer–writer conflicts need resolving, and those are comparatively rare.

That single property is why MVCC became dominant. PostgreSQL, Oracle, MySQL's InnoDB, SQL Server's snapshot mode, and essentially every modern system use it — and the reason is this paragraph, not any subtler advantage.

Each ROW VERSION carries:

```
xmin -- transaction that CREATED this version
xmax -- transaction that DELETED it (or none)
```

Each TRANSACTION takes a SNAPSHOT at start:

```
xmin_snapshot, xmax_snapshot, in_progress[]
```

A VERSION IS VISIBLE if:

```
xmin committed BEFORE the snapshot was taken, AND
xmax is unset, or committed AFTER the snapshot
```

Visibility is therefore computed per row, from the reading transaction's snapshot, with **no locks taken for reading at all**. A transaction sees a consistent picture of the database as it was at one instant, regardless of everything that has happened since. (The scheme above is PostgreSQL's; other systems differ in where versions are stored, not in the principle.)

2 Snapshot isolation

SNAPSHOT ISOLATION IS NOT SERIALIZABILITY

What MVCC naturally provides is **snapshot isolation**: each transaction reads from a consistent snapshot taken at its start, and write–write conflicts are resolved at commit time by first committer wins.

This prevents every anomaly on the SQL standard's list — dirty reads, non-repeatable reads, phantoms — and lost updates too.

And it is still not serializable, because, as Module 09 established, the standard's list is incomplete.

It permits write skew, which is not on the list, which real applications encounter, and which is the subject of the next page.

INVARIANT: at least one doctor must remain on call.
Initially Alice and Bob are both on call.

T1 (Alice): <pre>SELECT count(*) WHERE on_call -> 2, safe to leave UPDATE alice SET on_call = false COMMIT</pre>	T2 (Bob): <pre>SELECT count(*) WHERE on_call -> 2, safe to leave UPDATE bob SET on_call = false COMMIT</pre>
--	--

Both snapshots saw 2. Neither wrote a row the other wrote, so there is NO write-write conflict to detect. Both commit. Nobody is on call.

WHY SNAPSHOT ISOLATION CANNOT CATCH THIS

Each transaction's write is individually valid. At the moment T_1 decided, two doctors were on call, so Alice leaving was correct. The same was true for T_2 .

And there is no write–write conflict. T_1 wrote Alice's row; T_2 wrote Bob's. They touched disjoint data, so the first-committer-wins check finds nothing to complain about.

The conflict is between one transaction's read and the other's write — T_1 read Bob's row and T_2 wrote it, and vice versa. **Snapshot isolation does not track read–write dependencies at all**, so it cannot see the problem.

And no serial order produces this outcome. Run T_1 first and T_2 sees one doctor on call and does not leave. Run T_2 first and the same. The concurrent execution produced a state that no sequential execution could, which is precisely the definition of non-serializable.

SERIALIZABLE SNAPSHOT ISOLATION

Cahill, Röhm and Fekete, 2008. The insight is that a non-serializable execution under snapshot isolation always contains a particular structure: **two consecutive read–write dependencies** between concurrent transactions, forming part of a cycle.

So track read–write dependencies — not just the write–write ones snapshot isolation already checks — and abort a transaction when this 'dangerous structure' appears.

It remains optimistic: no locks are taken for reading, readers still never block, and the check happens at commit. The only cost is tracking the dependencies and some **false-positive aborts**, since the structure is necessary for a cycle but not sufficient.

PostgreSQL has implemented this as its SERIALIZABLE level since 9.1. It provides genuine serializability while keeping MVCC's non-blocking reads, which was widely assumed to be impossible, and it is one of the more satisfying results in the field.

3 Version storage and garbage

	Append-only (PostgreSQL)	Delta / undo (Oracle, InnoDB)
Where versions live	New versions are written into the table's own pages.	The current version stays in place; old versions go to a separate undo or rollback segment.
Table size	Grows with every update until vacuumed. Table bloat is the characteristic problem.	Stays compact.
Index maintenance	Every index must have an entry for every version , so an update touching one column still writes to every index. Expensive. (HOT updates mitigate this when no indexed column changes.)	Indexes point at the row's fixed location, so only indexes on changed columns are touched.
Reading an old version	It is in the table; visibility is checked in place.	Follow a chain through the undo log , applying deltas backwards. A long chain makes an old snapshot slow to read.
Characteristic failure	Bloat, and VACUUM not keeping up.	' Snapshot too old ' when the undo segment is exhausted and old versions have been discarded.

A LONG TRANSACTION PINS EVERY VERSION IN THE DATABASE

A version can only be reclaimed once **no running transaction could still need to see it** — which means once the oldest active snapshot is newer than the version's deletion.

So a single transaction left open for hours prevents the collection of every version created since it began — across the entire database, not merely the tables that transaction touched, because the collector cannot know what it might read next.

The table bloats, sequential scans read more pages, the buffer pool fills with dead rows, and nothing in the symptoms points at the cause. Performance degrades gradually across the whole system.

This is the most common MVCC operational problem by a wide margin, and the usual culprit is a connection sitting idle in transaction — an application that opened a transaction, did some work, and then went off to do something slow without committing. Monitoring for long-running transactions is not optional on an MVCC system.

- **Space.** Multiple versions of everything, until collected.
- **Garbage collection.** PostgreSQL's VACUUM, InnoDB's purge threads. This is real CPU and I/O, competing with the workload, and it must be tuned.
- **Index bloat** in append-only designs, where every version needs index entries.
- **A visibility check on every row read.** Small per row, and unavoidable, and it adds up on a large scan.
- **Transaction ID wraparound.** PostgreSQL's transaction counter is 32 bits. Because visibility is determined by comparing IDs, the counter wrapping would make old rows appear to be in the future and vanish. VACUUM must 'freeze' old rows before this happens, and **if it falls far enough behind, PostgreSQL refuses new transactions to protect the data.** This is a real production hazard, it has taken down large sites, and it is a direct consequence of the design choice in this section.

4 Choosing

	Two-phase locking	MVCC
Readers	Block writers on the same rows.	Never block anything.
Space overhead	Minimal — just the lock table.	Multiple versions of every updated row , plus collection.
Long-running reads	Catastrophic — they hold locks for their whole duration.	Harmless to writers; harmful only through delayed garbage collection.
Write conflicts	Detected when the lock is requested.	Detected at commit — so work may be discarded.
Default isolation	Serializable is achievable at reasonable cost.	Usually snapshot isolation , with true serializability available at extra cost.
Used by	DB2 and older systems.	PostgreSQL, Oracle, MySQL InnoDB, SQL Server snapshot mode.

OPTIMISTIC CONCURRENCY CONTROL, THE THIRD OPTION

Run the transaction with no locks at all, recording what was read and what was written. At commit, validate: has anything this transaction read been modified since it was read? If so, abort and retry; otherwise, commit.

Excellent when conflicts are rare. No locking overhead, no lock table, no deadlock detection, and no blocking of any kind.

Terrible when conflicts are common. Work is performed and then discarded, repeatedly, and under heavy contention throughput can collapse — transactions spend all their time doing work that is thrown away, and the situation is self-reinforcing because retries add load.

It is used in in-memory systems, where transactions complete in microseconds so the conflict window is tiny, and in application-level patterns — the 'version column' check in ORMs is optimistic concurrency control under another name.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lecture 19, Multi-Version Concurrency Control	Version storage, visibility, and garbage collection across real systems.

Resource	What it covers for this module
Wu et al. — An Empirical Evaluation of In-Memory MVCC (free)	A systematic comparison of version storage schemes and their costs. The source for §3's table.
Cahill, Röhm & Fekete — Serializable Isolation for Snapshot Databases (free)	SSI, which became PostgreSQL's SERIALIZABLE. The dangerous-structure argument is clearer in the original.
PostgreSQL documentation — Concurrency Control and Routine Vacuuming	The implementation in §1 and §3, including wraparound and what to monitor.
Kleppmann — Designing Data-Intensive Applications, chapter 7	Write skew explained with more real examples, including ones that caused incidents.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement MVCC visibility checking with xmin, xmax, and snapshots. Verify a reader sees a consistent snapshot while writers proceed.
- 2 Demonstrate non-blocking reads: start a long read and confirm writers are not delayed. Repeat under 2PL and measure the difference.
- 3 **Reproduce the write skew example** in PostgreSQL at REPEATABLE READ. Then set SERIALIZABLE and show one transaction aborts.
- 4 Construct two more write skew scenarios from an application domain you know.
- 5 Measure table bloat: update one column of a million-row table a hundred times and report the table size before VACUUM and after.
- 6 Open a transaction and leave it idle. Run a heavy update workload and measure the bloat the idle transaction causes.
- 7 Query pg_stat_activity for long-running transactions and write a monitoring query you would actually deploy.
- 8 Measure the false-positive abort rate of SSI on a workload that is genuinely serializable.
- 9 Implement optimistic concurrency control and measure throughput against contention level, identifying where it collapses.
- 10 Compare 2PL, snapshot isolation, and SSI on the same workload: throughput, abort rate, and the anomalies each permits.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is MVCC's central property, and why did it win?
- 2 How is version visibility determined, and what locks are involved in reading?
- 3 What does snapshot isolation prevent, and why is it still not serializable?
- 4 Work through the write skew example and explain why no conflict is detected.
- 5 What does serializable snapshot isolation track, and what does it cost?
- 6 Compare append-only and delta version storage on five axes.
- 7 Why does one long transaction cause database-wide bloat?
- 8 Give five costs of MVCC, including the one that can halt a database.
- 9 When is optimistic concurrency control appropriate, and when does it collapse?